Oluwatosin Ayeni

Glasgow, Scotland, UK
07727305230 · ayenioluwatosinolawale@gmail.com
github.com/AyeniOluwatosinOlawale · linkedin.com/in/oluwatosinolawaleayeni

30 October 2026

Doubleword, London, UK

Dear Hiring Team,

I am writing to apply for the Senior Infrastructure Engineer (LLM Inference Systems) role at Doubleword. Designing and operating the infrastructure layer that keeps a GPU inference cluster running at throughput — batch job scheduling, request routing, SLO compliance under load — is exactly the systems problem I have been building toward independently.

Three achievements directly relevant to what this role requires:

- **Production LLM Inference Platform on Kubernetes:** I built *llm-inference-platform* — vLLM + SGLang + LiteLLM gateway deployed on RunPod and Vast.ai GPU cloud, with KEDA event-driven autoscaling, Prometheus/Grafana/Loki/AlertManager observability, and Velero backup. The platform handles multi-tenant request routing and GPU pool management — a complete production inference infrastructure stack. GitHub
- **H100 GPU cluster operations and SLO compliance:** I benchmarked and operated vLLM v0.30 and SGLang v0.5.20 on 2× NVIDIA H100 NVL 94 GB (NVLink 4.0, CUDA 13.4), measuring throughput, TTFT, TPOT, and SLO compliance across concurrency 1–128. Peak: 5,965 tok/s; 100% SLO compliance at concurrency 128. RadixAttention prefix caching: 1,443× TTFT speedup at 131K-token context. This is the level of rigor I apply to serving infrastructure decisions. Live results.
- **KV-cache-aware routing and GPU profiling:** I designed KV-cache-aware request routing for vllm-k8s-stack — routing decisions account for prefix reuse so cache state is not wasted across GPU nodes. I also profiled CUDA KV cache ops with NVIDIA Nsight Systems and Compute, identified a memory bandwidth bottleneck, and retuned kernel access patterns for a 3.8× throughput improvement. Infrastructure-level thinking, not just configuration.

I am based in Glasgow and open to London 2–3 days/week. I write primarily in Python with active Rust tooling work; I understand why low-latency infrastructure demands Rust for the hot path.

Yours sincerely,

Oluwatosin Ayeni

github.com/AyeniOluwatosinOlawale
linkedin.com/in/oluwatosinolawaleayeni